📝 AI UI Model Specification: Property & Tenant SaaS

1 Goal

The AI should generate **responsive, scalable, and production-ready frontend code** for a property/tenant management SaaS platform that supports:

- Multi-property landlords
- Multi-room properties (hostels, flats, houses)
- Tenant onboarding, lease management, payments, inspections, and notifications
- Admin dashboards for landlords and central superadmins
- Highly dynamic UI for custom rules (deposits, late fees, notices, etc.)

The frontend should be **modern, accessible, and mobile-first**, able to integrate with a FastAPI backend via REST/GraphQL.

2 Target Users / Personas

Persona	Description	Key Needs
Landlord / Property Manager	Owns multiple properties, may manage 5–200+ units	Quick overview of payments, tenants, leases; rule management
Tenant	Single/multi-room tenants	Onboarding, contract signing, rent tracking, document submission
Super Admin	Manages platform centrally	Analytics, landlord monitoring, audit logs

3 Functional UI Requirements

1. **Authentication & Onboarding**
   - Login / Signup
   - OAuth (Google, Email)
   - Passwordless onboarding link for tenants
   - Document upload (PDF, images)
   - Contract signing (e-signature)
2. **Dashboard / Overview**
   - Property summary (units, occupancy, pending payments)
   - Tenant activity timeline
   - Financial summary (rent collected, deposits, late fees)
3. **Property & Units Management**

- Add / Edit / Delete properties and units
- Customizable rules per property (rent, deposit, notice period, late fees)
- Bulk operations for multi-room properties

4. **Tenant Management**
   - Invite tenants via link
   - View onboarding progress
   - Lease status (Draft / Pending / Active / Notice / Closed)
   - Payment history
   - Deposit tracking
5. **Payment & Finance**
   - Rent payment tracker
   - Late fee calculation
   - Deposit ledger
   - Exportable reports
6. **Workflow / Inspections**
   - Schedule inspections
   - Capture and preview images
   - Inspection approval
   - Move-in / move-out checklist
7. **Notifications**
   - WhatsApp / Email / SMS integration
   - Template-based message preview
   - Sent / delivered / failed status
8. **Rules & Customization**
   - Dynamic UI for setting rules per property
   - Validation for conflicting rules
   - Conditional forms based on property type or workflow stage
9. **Analytics**
   - Occupancy rates, pending rent, cash flow
   - Filter by property, unit type, tenant

---

4 Non-Functional UI Requirements

- **Responsive & mobile-first:** Desktop + tablet + mobile
- **Performance:** Lazy-loading for large lists (100+ rooms)
- **Accessibility:** WCAG 2.1 compliance
- **Theming / Branding:** Light/dark mode, color customization
- **State Management:** Support offline draft saves (Redux, Zustand, or equivalent)
- **Localization-ready:** Support multiple languages
- **Security:** Input validation, XSS protection, secure document upload

---

5 Technical Requirements

- **Framework:** React / Next.js preferred
- **UI Library:** TailwindCSS + shadcn/ui or Material UI
- **Form Handling:** React Hook Form / Formik
- **Data Fetching:** React Query / SWR
- **State Management:** Zustand / Redux
- **File Uploads:** S3-compatible API or signed URL
- **Charts:** Recharts or ApexCharts for analytics
- **Routing:** Next.js app-router, dynamic routes for properties, units, tenants
- **Component Modularity:** Each module (leases, tenants, payments) has reusable components